

Characterizing Network Traffic with Programmable Dataplanes

Abstract—This short homework serves to assess the programming, statistical analysis, and networking capabilities of candidates for doctoral positions in the Network Data Science group at IMDEA Networks Institute. The homework is aligned with the research interests of the group, which involve experimental networking research, traffic analysis, and programmable dataplanes. However, it shall be regarded as a simple exercise, which is not necessarily representative of the more interesting (and complex) subjects that will be explored during the doctoral theses in the group.

I. TASKS

The homework is about analyzing a dataset of real-world network traffic recorded in the upstream ISP link of a research institute—The MAWI (Measurement and Analysis on the WIDE Internet) Working Group [1]. In particular, the traffic record (PCAP file) corresponds to 15 minutes of traffic. Note that packets are truncated after 96-bytes and IP addresses have been scrambled to preserve privacy. Using this dataset, the candidate is asked to complete the three tasks described next.

Task I. Software-based Traffic Characterization and Statistical Analysis. In the first task, the applicant has to perform a basic characterization of the data using a software tool like `tshark` or another tool they deem appropriate. Specifically, the following results should be produced as a minimum:

- *Feature Extraction & Plotting:* A plot of the probability distribution functions (PDFs) of features such as packet size, inter-arrival times (IAT), or more complex flow-level features such as duration or throughput.
- *Statistical Modeling:* Fit a distribution to any of the features gathered (e.g., a Poisson distribution for the IAT) and evaluate the goodness of fit.

Additionally, the applicant might break down the analysis by protocol (e.g., TCP vs UDP) or by flow characteristics (e.g., short vs long flows) to provide a more detailed characterization of the traffic.

Task II. In-Network Feature Extraction using P4. In the second task, the applicant is asked to write a simple P4 program for the `bmv2` target to capture the same features analyzed in Task I. The methods should yield comparable results.

- *Packet-level Features:* For features like IAT¹, the applicant can use a single-entry stateful register to store the

¹Since `bmv2`'s performance is very low compared to HW targets like the Intel Tofino, it is recommended to send the trace at a slower rate. For this, `tcpdump`'s `--multiplier=value` option might come handy.

timestamp of the last packet, compute the IAT, and store it in a custom header field.

- *Flow-level Features:* For flow-level features, the applicant can use counters/registers to track flows, read their values via the `gRPC` interface that the `simple_switch_grpc` exposes, and process the output in Python.

Task III. Comparison and Discussion. Finally, the applicant should compare the results obtained from both methods (software-based and in-network) and discuss the benefits and drawbacks of each approach. This discussion should consider aspects such as accuracy, overhead, scalability, and ease of implementation.

II. METHODS

- **Reference Dataset:** The reference dataset for the homework is a daily trace² from the MAWI Working Group Traffic Archive [1]. For development and testing purposes, applicants can use a smaller subset of the trace (e.g. first 100 flows), which can be obtained by using tools like `tcpdump` to extract a portion of the original PCAP file. However, the final results should be obtained using, at least, the first 10,000 packets of the trace.
- **Tools and Environment:** Applicants may employ any programming language or combination of tools they deem appropriate to solve Task I (Python is recommended). For Task II, applicants must use the P4 language targeted for the `bmv2` software switch. Instructions on how to set up the `bmv2` environment and run P4 programs can be found in the P4 tutorial repo [2]. Applicants are encouraged to use the `simple_switch_grpc` target, which allows for easy interaction with the P4 program via `gRPC` instead of the CLI, which is more suitable for this task. Additional resources for learning P4 and the `bmv2` target can be found in the P4 documentation [3], and `p4-guide` [4].
- **Use of AI:** Employing AI (e.g., LLMs) to complete the tasks is allowed as long as the deliverables clearly state which tools were used, how they were leveraged to produce code and/or results, and which kind of verification was run by the applicant; using AI-assisted tools to write entirely or support the writing of report sections of the deliverables is **not** allowed.

²<http://mawi.nyu.edu/mawi/samplepoint-F/2013/201302011400.dump.gz>

III. DELIVERABLES

The applicant is required to submit the following documents by the agreed deadline:

- **Jupyter Notebook:** A cohesive notebook containing the distribution plots, the statistical fitting results, and a brief discussion analyzing the benefits and drawbacks of the software-based method (Task I) versus the programmable dataplane method (Task II).
- **Source Code Archive:** Either the URL of a Git repository, or a single and well-organized archive. Both should include (i) all source code for both methods, and (ii) precise instructions on how to run both methods on a standard Linux environment.

For any doubt or question, contact marco.fiore@imdea.org.

REFERENCES

- [1] MAWI Working Group Traffic Archive, [Online]. Available: <https://mawi.wide.ad.jp/mawi/>
- [2] P4 Language Tutorials, [Online]. Available: <https://github.com/p4lang/tutorials>
- [3] P4 Language Specification and Documentation, [Online]. Available: <https://p4.org/p4-spec/docs/>
- [4] P4 Guide Repository, [Online]. Available: <https://github.com/jafingerhut/p4-guide>